

Manuel de développement

Organisation du projet

Le projet est structuré de manière modulaire afin de garantir une séparation claire des responsabilités et une meilleure lisibilité du code.

Le code source est regroupé dans le dossier **src**, qui contient l'ensemble des modules nécessaires au fonctionnement du jeu.

Architecture générale

Le dossier **src** est organisé autour de trois modules principaux : **game**, **loader** et **model**, auxquels s'ajoute un dossier **data** contenant les ressources du jeu.

Module **game**

Le module **game** contient l'ensemble du code assurant le bon fonctionnement du jeu. Il constitue le cœur de l'application et gère la logique principale ainsi que l'affichage.

Ses responsabilités principales sont :

- la gestion des interactions du joueur, notamment les clics
- la modification des valeurs du jeu en fonction des actions du joueur
- la gestion de l'inventaire du joueur
- la gestion des combats
- la gestion de l'affichage graphique du jeu

Ce module centralise la coordination entre les actions du joueur, l'état du jeu et son rendu visuel.

Module **loader**

Le module **loader** regroupe tous les éléments qui doivent être chargés ou préparés avant le lancement du jeu.

Il permet d'isoler les opérations de chargement afin d'avoir de l'optimisation par la suite. Il est notamment responsable du :

- chargement des polices d'écriture
- chargement des images utilisées dans le jeu

- le calcul préalable de la position et des dimensions des images en fonction de la taille de l'écran

Ces calculs anticipés permettent d'optimiser l'affichage et d'adapter le jeu à différentes résolutions d'écran.

Module **model**

Le module **model** contient l'ensemble des classes liées à l'initialisation et à la représentation des éléments du jeu.

Il se concentre exclusivement sur la logique métier et ne gère ni l'affichage graphique ni les interactions directes avec le joueur.

Ses responsabilités incluent :

- l'initialisation des objets
- l'initialisation de la carte du jeu ainsi que de ses salles
- l'initialisation des monstres
- l'initialisation du sac du joueur
- l'initialisation du héros
- la gestion des effets appliqués durant les combats

Dans ce module, il nous a paru logique d'avoir une interface pour les salles, les ennemies, ainsi que pour les items sachant que les items/salles/ennemies ont le même comportement.

Dossier **data**

Le dossier **data** contient l'ensemble des ressources graphiques utilisées par le jeu. Il regroupe notamment toutes les images nécessaires à l'affichage des éléments du jeu (personnages, objets, interfaces, etc.).

Améliorations et correctifs depuis la soutenance

Depuis la soutenance, plusieurs améliorations et correctifs ont été apportés au projet afin d'améliorer la qualité du code, sa lisibilité et sa robustesse :

- utilisation de records pour les armes à la place de classes classiques
- utilisation d'itérateurs pour parcourir les listes
- la taille du sac ainsi que la taille de la map peut être changer (pas codé en brut)
- suppression des méthodes **default** dans les interface
- utilisation de Stream

- suppression des attributs `public static`
- ajout de `copyOf` afin d'éviter la modification des listes
- passage d'interfaces en paramètres au lieu d'implémentations concrètes (List et non ArrayList)
- séparation claire entre les items et leur affichage (par exemple la description)
- nous n'avons plus la création inutile de nouveaux tableaux avec `next()`.
(Direction)

Ces améliorations permettent de respecter de meilleures pratiques de développement, de renforcer l'encapsulation et de rendre le code plus maintenable et évolutif.